# Fundamentals of Machine Learning

# IT5022

# Programming Assignment




**Group Members:**

**MS26923024 – Saara M.K.F**

**MS26924250 – Dhanushikan V**

# Contents

1. Problem and Dataset Description ..... 3
2. Data Exploration and Preprocessing ..... 4
3. Algorithm Choice and Evaluation ..... 7
4. Results: Random-Split Classification ..... 10
5. Results: Generalization to Later Batches ..... 13
6. Critical Discussion, Limitations and Future Improvements ..... 15
7. Individual Contribution ..... 16
8. References ..... 16
9. Appendix A - Complete Source Code ..... 17

## 1. Problem and Dataset Description

Metal-oxide gas sensors respond differently to volatile compounds. Over time, their readings can change as sensors age or acquisition conditions vary [2].

**This project studies two questions: (1) how accurately six gases can be identified from sensor-response measurements, and (2) how reliably the trained models perform on data collected in later periods.**

The Gas Sensor Array Drift at Different Concentrations dataset is a published research **dataset containing 13,910 measurements collected across ten acquisition batches over approximately 36 months. Each observation has 128 numerical features** (16 sensors × eight extracted response attributes). The prediction target is one of six gases: acetaldehyde, acetone, ammonia, ethanol, ethylene or toluene. The dataset contains no reported missing feature values. Batch and concentration are kept only for analysis but never used as predictors. [1,2]

Each sensor's eight features **comprise two steady-state descriptors**: the **maximal resistance change ΔR** and **its normalized form |ΔR|** - and six transient descriptors computed as exponential moving averages of the rising and decaying response at three smoothing constants ($\alpha = 0.1, 0.01, 0.001$).

This is a substantially more complex dataset than introductory examples, which makes dimensionality reduction and chronological testing meaningful. Batch order is taken from the original filenames, and the per-batch counts were checked against the published documentation. [1,2]

| **Characteristic** | **Value** | **Why it matters** |
|---|---|---|
| Dataset Source | UCI ML Repository, ID 270 | Public dataset with a primary research citation |
| Size | 13,910 rows; 128 features | Sufficient dimensionality to evaluate PCA |
| Task | Six-class supervised classification | Two distinct learning algorithms required |
| Time structure | 10 chronological batches | Allows early-to-late generalization analysis |
| Metadata | Concentration and batch ID | Used to examine possible confounding only |

## 2. Data Exploration and Preprocessing

The gas classes are moderately uneven (approximately 1,641 - 3,009 observations per class), and batch sizes differ considerably. **We inspected class counts, batch composition, feature magnitudes, concentration and correlation**. These checks motivated standardized features and macro-averaged classification metrics.

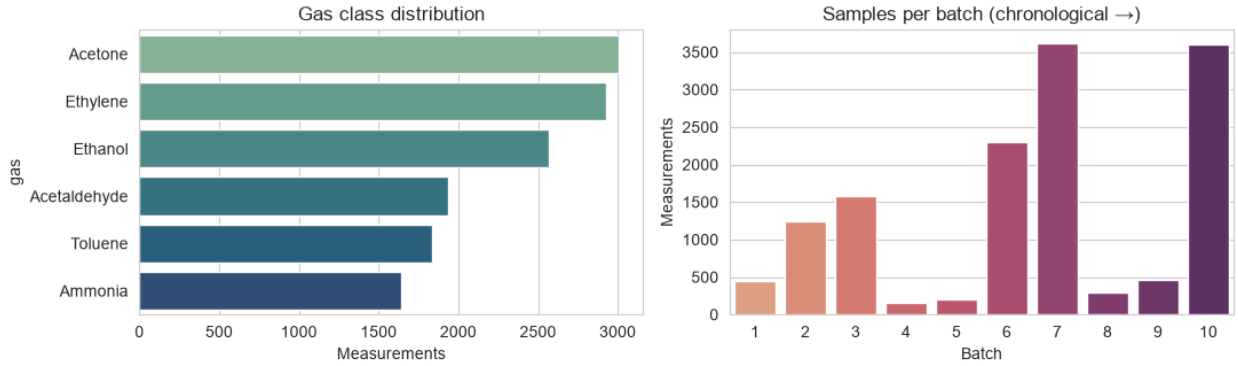



Figure 1. Gas-class counts and sample counts across the ten acquisition batches.

Because the chronological study trains on early batches and tests on later ones, the class composition over time matters: a change in accuracy could reflect a change in which gases appear rather than sensor ageing alone.

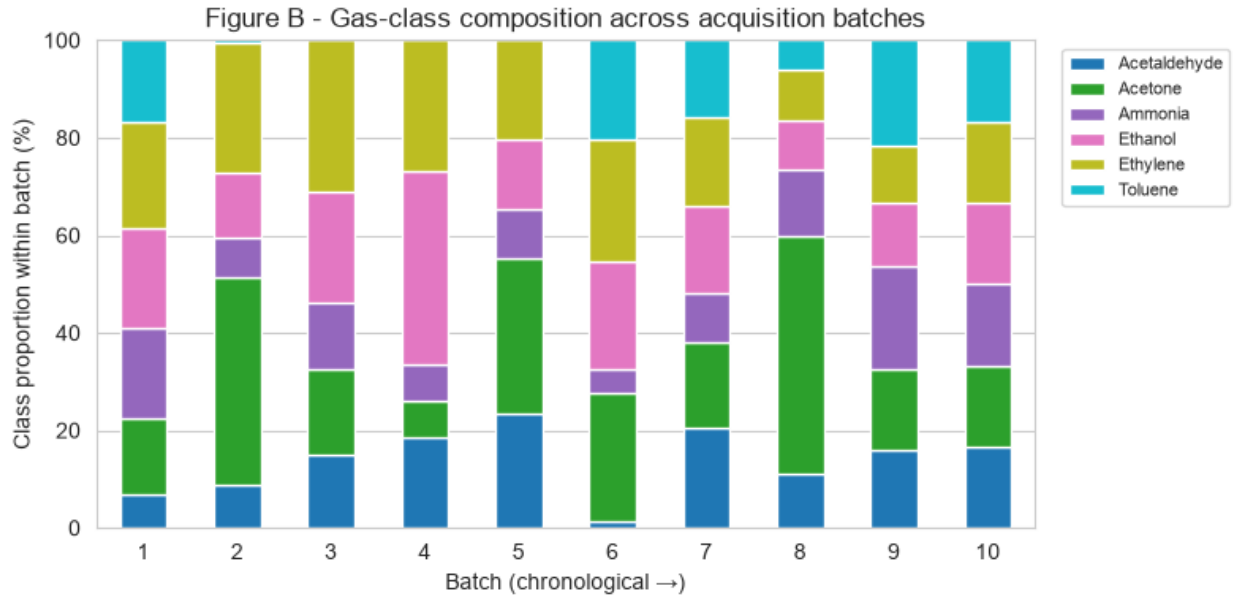



Figure 2. Gas-class proportions vary across acquisition periods; some gas types are missing from particular batches.

Concentration is retained only as metadata and is never a predictor (X = df[FEATURES]). Its distribution differs across batches - batch 6, for instance, is markedly lower - which is a further confounder for the drift analysis.

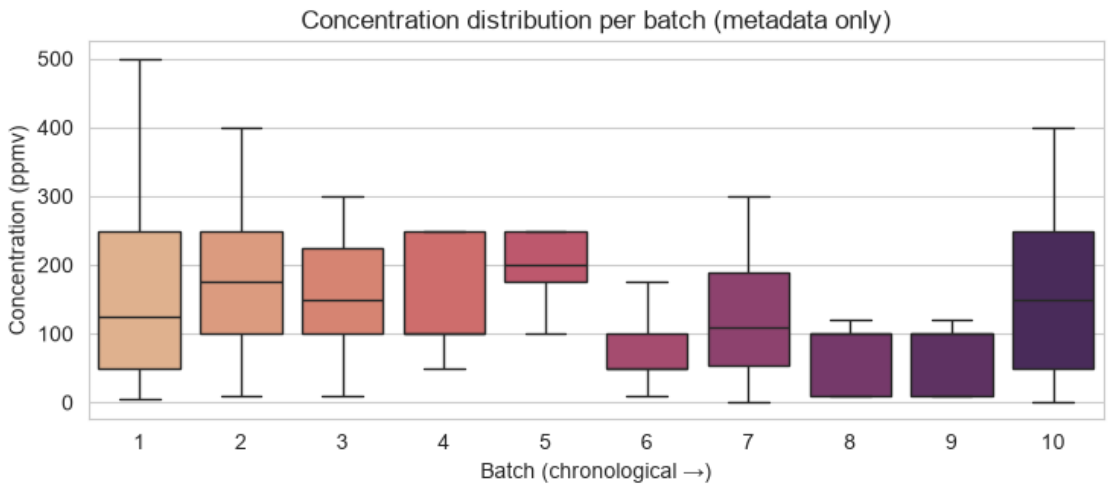



Figure 3: Concentration distribution per batch (metadata only, excluded from the model inputs).

## 2.1 Standardization

The raw features span about six orders of magnitude. StandardScaler is fitted on training data only, and inside cross-validation every learned transformation is re-fitted independently within each fold [4]. On a leakage-free training comparison, standardization raised Logistic Regression CV accuracy from roughly 0.978 to 0.989; the unscaled run produced convergence warnings, so that baseline is not fully converged.




Figure 4. Correlations among the first 24 sensor features motivate examining redundancy and dimensionality reduction.

### 2.2 Feature Engineering and Selection

An alternative representation produced 176 engineered features: 128 signed-log transforms, 16 per-sensor means, 16 ranges and 16 cross-sensor ratios. These aggregates are mathematical summaries whose physical meaning is limited. Training-only ANOVA F-statistics and mutual information were examined for relevance, and SelectKBest (k = 40) was fitted inside each cross-validation fold when evaluating the engineered representation [4]. Consistent with the module's lab material, we treat statistical significance as distinct from predictive usefulness.

### 2.3 PCA Dimensionality Reduction

Principal Component Analysis projects the standardized data onto directions of decreasing variance. For standardized inputs X and selected principal directions W the transformed data is $Z = X \cdot W$ [6]. On the training split, 13 components retain at least 95% of the variance, compressing the 128 inputs considerably. Retained variance is not the same as predictive accuracy - 95% variance does not imply 95% accuracy.

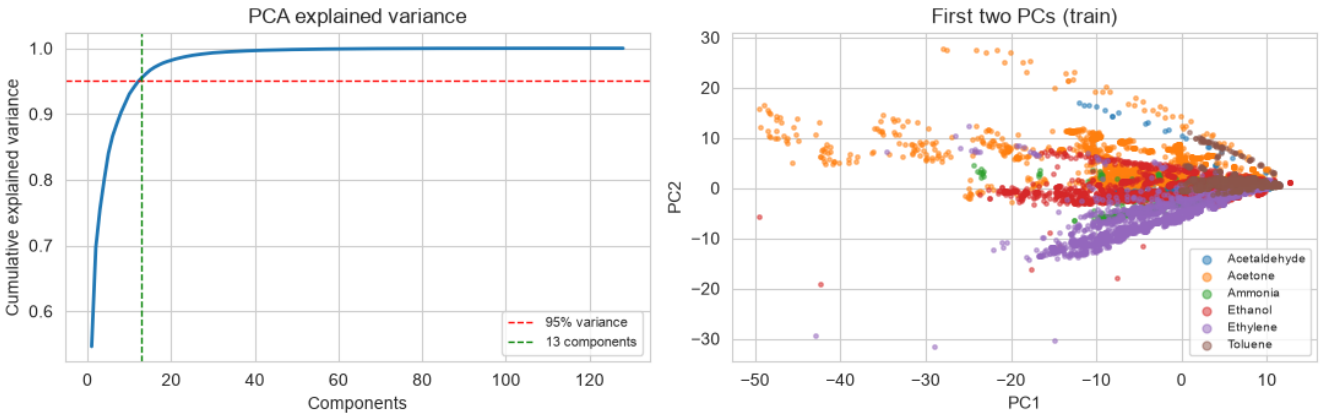



Figure 5. PCA cumulative explained variance, reaching 95% at 13 components (left), and the first two principal components (right).

## 3. Algorithm Choice and Evaluation

The assignment requires two distinct supervised algorithms. We chose Logistic Regression an interpretable linear multiclass model that fits class probabilities with regularization and Random Forest an ensemble of decision trees that captures non-linear structure through bagging and random feature subsets. Their contrasting assumptions support an informative comparison, and Logistic Regression coefficients together with Random Forest importances give two interpretability views. [4,5]

**Logistic Regression** models each class with a **softmax probability**

$$P(y = k \mid x) = \frac{exp(\mathbf{w}_k^\top \mathbf{x} + b_k)}{\sum_j exp(\mathbf{w}_j^\top \mathbf{x} + b_j)}$$

minimizing multiclass cross-entropy; the inverse regularization strength C controls over-fitting.

**Random Forest** grows many trees that each split to reduce node impurity (**Gini impurity**)

$$G = 1 - \sum_k p_k^2$$

and averages their votes, which lowers the variance of a single tree.

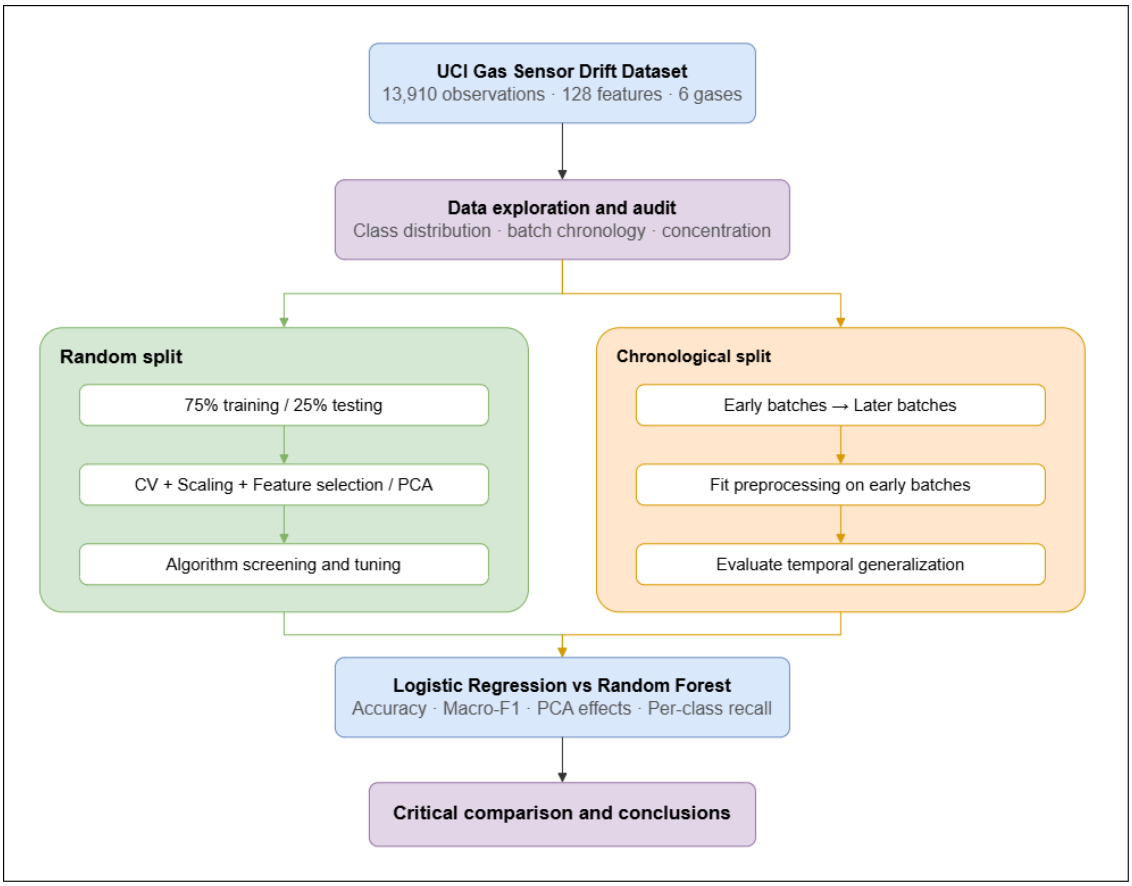



Figure 6. Algorithm Choice and Evaluation

### 3.1 Algorithm Screening

Before fixing the pair, six classical classifiers were screened with 3-fold stratified cross-validation on the training data, under two representations the original 128 features and PCA(95%) with all preprocessing kept inside the pipelines. This was an exploratory model-selection screen, not a comprehensive hyper-parameter search.

| Algorithm | Original CV macro-F1 | PCA CV macro-F1 |
|---|---|---|
| Random Forest | 0.9921 | 0.9877 |
| K-Nearest Neighbors | 0.9882 | 0.9828 |
| Logistic Regression | 0.9882 | 0.9082 |
| SVM (RBF) | 0.9768 | 0.9296 |
| Decision Tree | 0.9578 | 0.9584 |
| Gaussian Naïve Bayes | 0.5690 | 0.6258 |

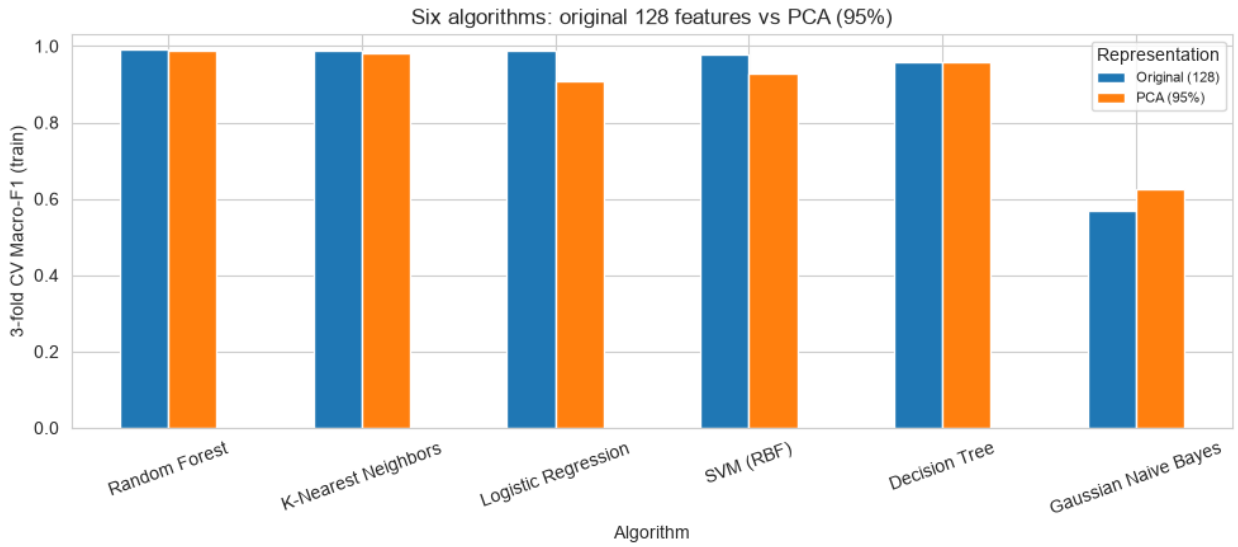



Figure 7. Six-algorithm screen: training-CV macro-F1 on the original 128 features vs PCA(95%).

Random Forest ranked highest under both representations and was selected. Logistic Regression was retained because it is highly competitive on the original features (0.9882, tying K-Nearest Neighbors) and provides a simpler, interpretable linear baseline; the choice was not based on absolute ranking alone. The effect of PCA is strongly algorithm-dependent: the linear and margin models (Logistic Regression, SVM) lose the most, the tree models are barely affected, and Naïve Bayes actually improves consistent with PCA removing the linear correlation among features, a partial step toward the independence it assumes (decorrelation is not full statistical independence).

### 3.2 Evaluation Design

Random-split evaluation uses a stratified 75/25 split; model/representation selection and tuning are performed by cross-validation on the training portion, and the held-out split provides the final random-split estimate. Because this split was examined during iterative development, those scores are best read as in-distribution estimates rather than untouched prospective results. Chronological evaluation trains on earlier batches and tests on later ones, with three overlapping splits that probe sensitivity to the training period; these are temporal sensitivity analyses, not independent deployment trials.

The primary measures are accuracy and macro-F1, complemented by balanced accuracy, per-gas recall, confusion matrices and one-vs-rest multiclass ROC-AUC. Macro-F1 weights every gas equally, which is important because class frequencies differ by batch. Hyper-parameters for the PCA variants were chosen with training-only GridSearchCV (3-fold): Logistic Regression $C \in \{0.1, 1, 10\}$; Random Forest 100 or 200 trees, max depth {None, 20}, minimum leaf size 2.

## 4. Results: Random-Split Classification

Cross-validation showed that the original 128 features gave the best macro-F1 for both selected algorithms; the engineered-and-selected features did not improve performance, and PCA affected Logistic Regression more than Random Forest.

| Model | Original (128) | PCA (95%) | Engineered + Select |
|---|---|---|---|
| Logistic Regression | 0.9882 | 0.9083 | 0.9705 |
| Random Forest | 0.9920 | 0.9877 | 0.9898 |

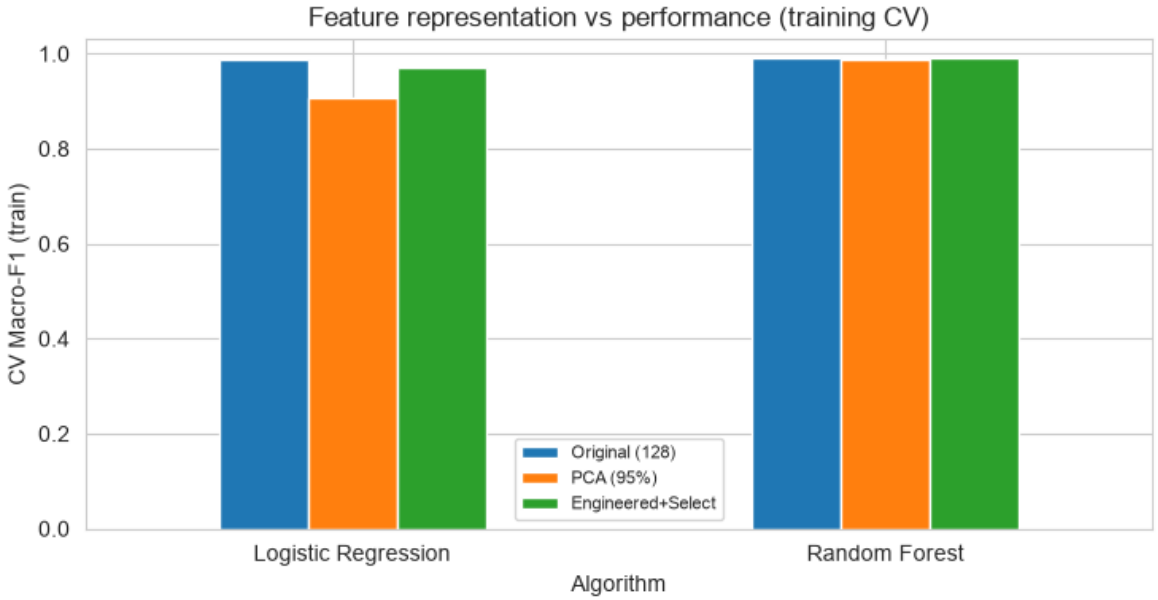



Figure 8. Feature-representation comparison (training-CV macro-F1).

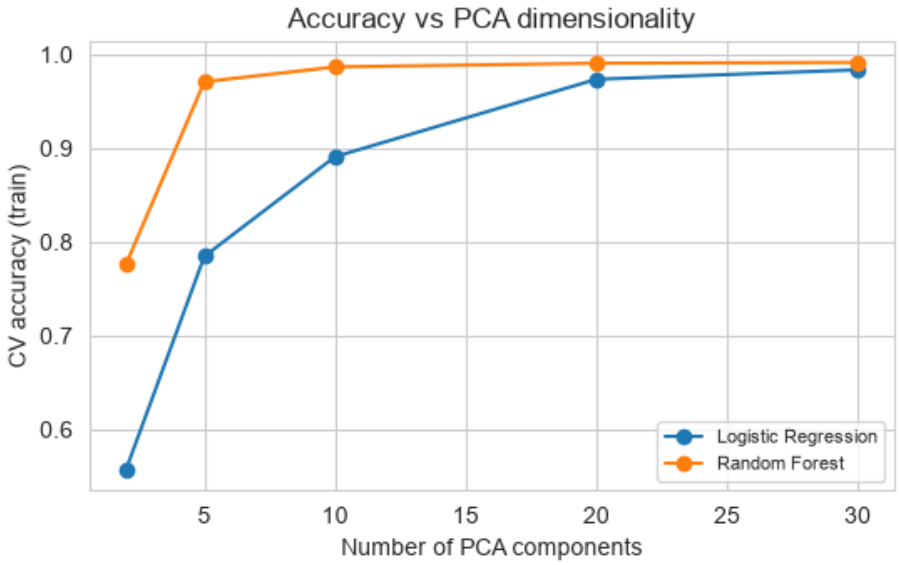



Figure 9. Classification accuracy versus the number of PCA components (training CV).

Both models were tuned with training-only GridSearchCV (best CV macro-F1: Logistic Regression 0.909 at C = 10; Random Forest 0.989 at max_depth = 20), and the refit best estimators were evaluated on the held-out test set. Two objectives are reported: the best predictive representation (original 128 features) and the mandated PCA reduction (tuned).

| Representation | Model | Accuracy | Macro-F1 | Balanced Acc. | ROC-AUC(ovr) |
|---|---|---|---|---|---|
| All (128) | Logistic Regression | 0.9917 | 0.9915 | 0.9915 | 0.9984 |
| All (128) | Random Forest | 0.9934 | 0.9931 | 0.9929 | 0.9994 |
| PCA (tuned) | Logistic Regression | 0.9362 | 0.9298 | 0.9311 | 0.9915 |
| PCA (tuned) | Random Forest | 0.9914 | 0.9910 | 0.9906 | 0.9998 |

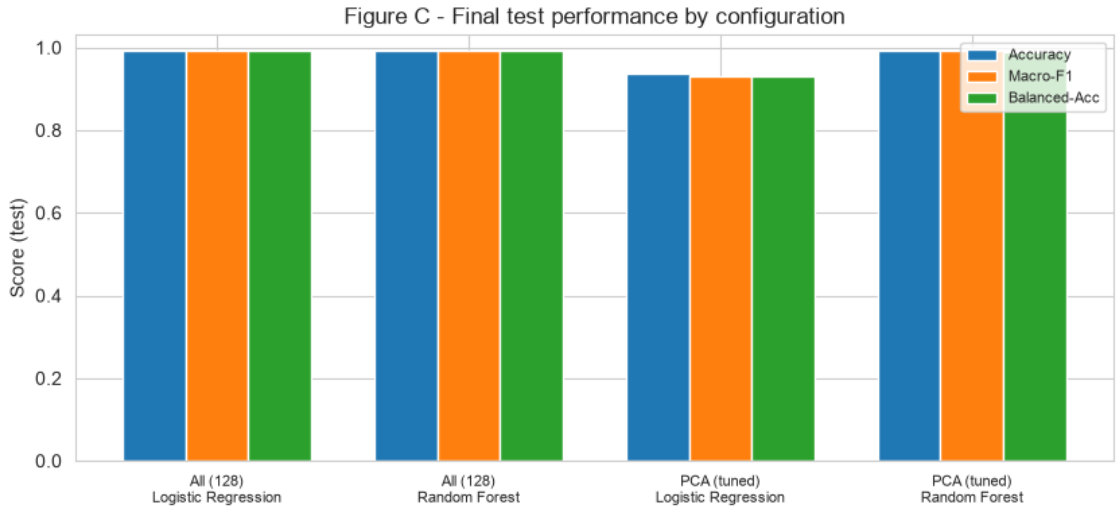



Figure 10. Final held-out test performance by configuration.

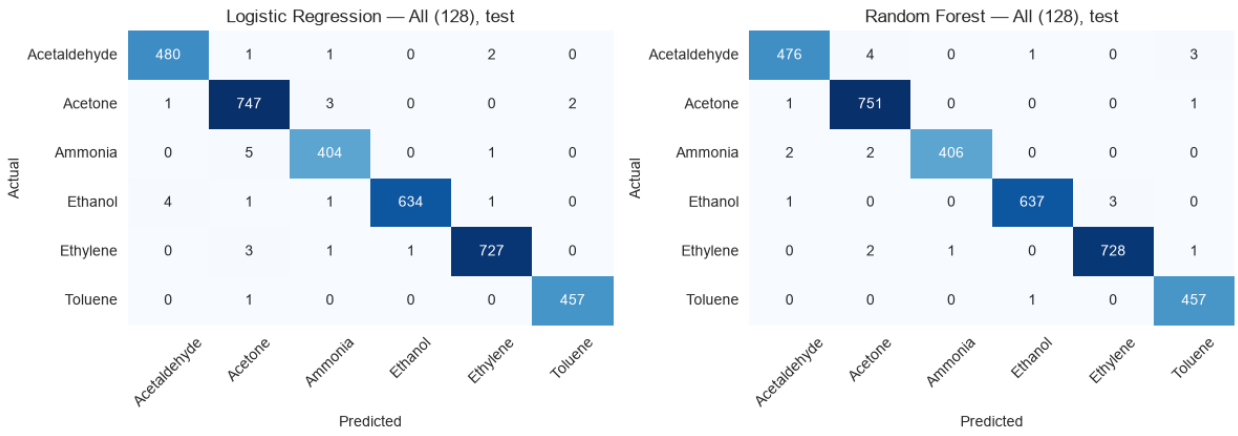



Figure 11. Held-out confusion matrices with the original features: Logistic Regression (left) and Random Forest (right).

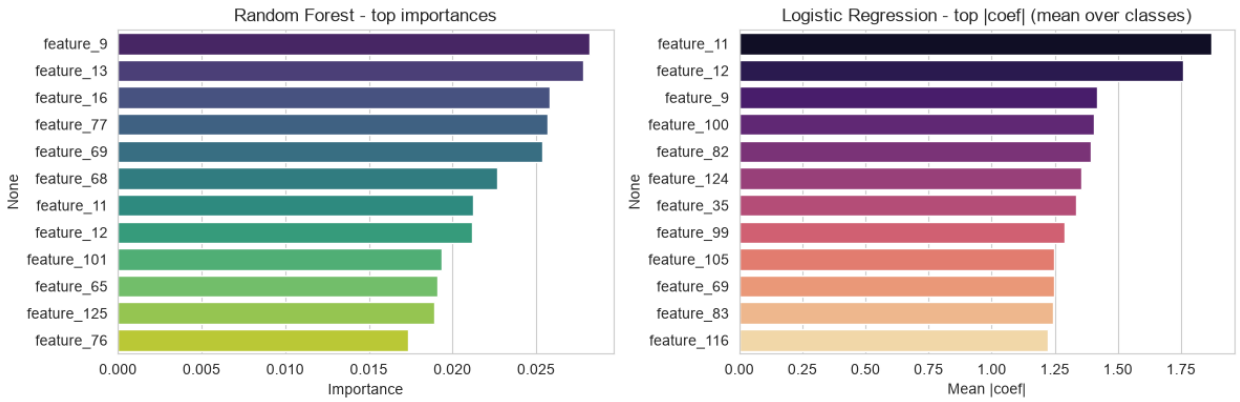



Figure 12. Random Forest feature importances (left) and mean absolute Logistic Regression coefficients (right).

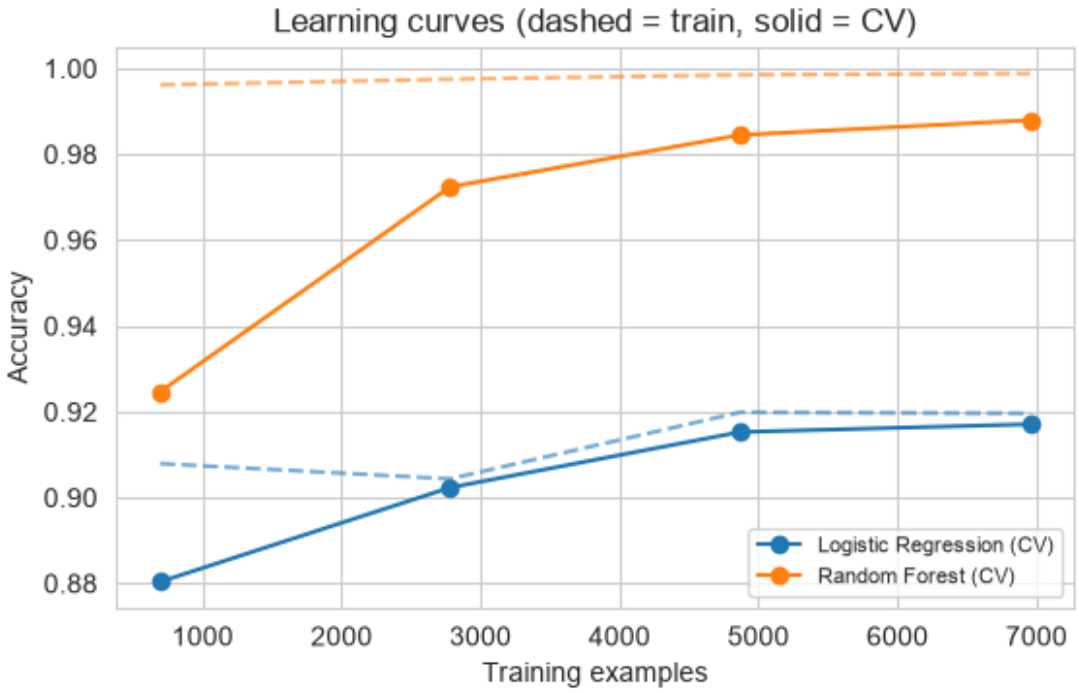



Figure 13. Learning curves (dashed = training score, solid = cross-validation score).

## 5. Results: Generalization to Later Batches

The random-split results are very strong, but evaluation on future acquisition batches is substantially more difficult. The chronological experiments below fit preprocessing and classification on early batches only. Their lower results can reflect sensor changes, but also differences in gas and concentration coverage across batches.

| Early → Later | LR acc. | LR macro-F1 | RF acc. | RF macro-F1 |
|---|---|---|---|---|
| Train 1-5 / test 6-10 | 0.6252 | 0.5895 | 0.5569 | 0.4936 |
| Train 1-3 / test 4-10 | 0.5945 | 0.5166 | 0.5879 | 0.5242 |
| Train 1-6 / test 7-10 | 0.6042 | 0.5866 | 0.6189 | 0.6071 |

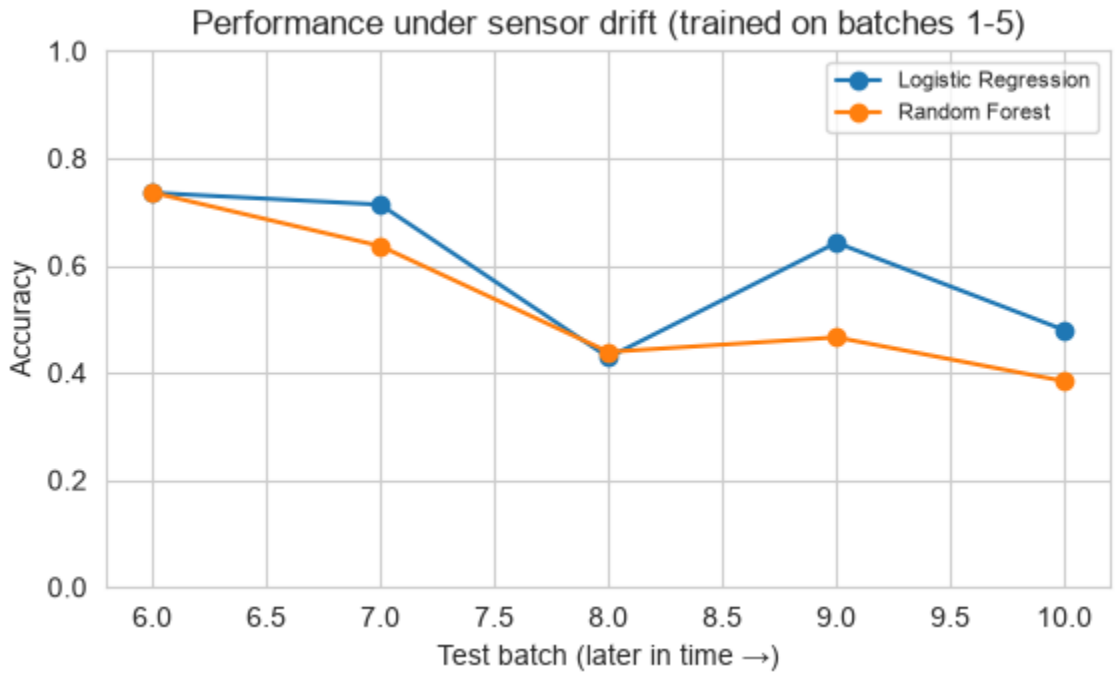



Figure 14. Later-batch accuracy of models trained on batches 1–5.

Logistic Regression has higher accuracy in two chronological configurations and Random Forest in the third, so neither is consistently superior under temporal change. The train-versus-future gap is consistent with a mix of shifting measurement distributions and possible overfitting; the experiment does not isolate a single cause.

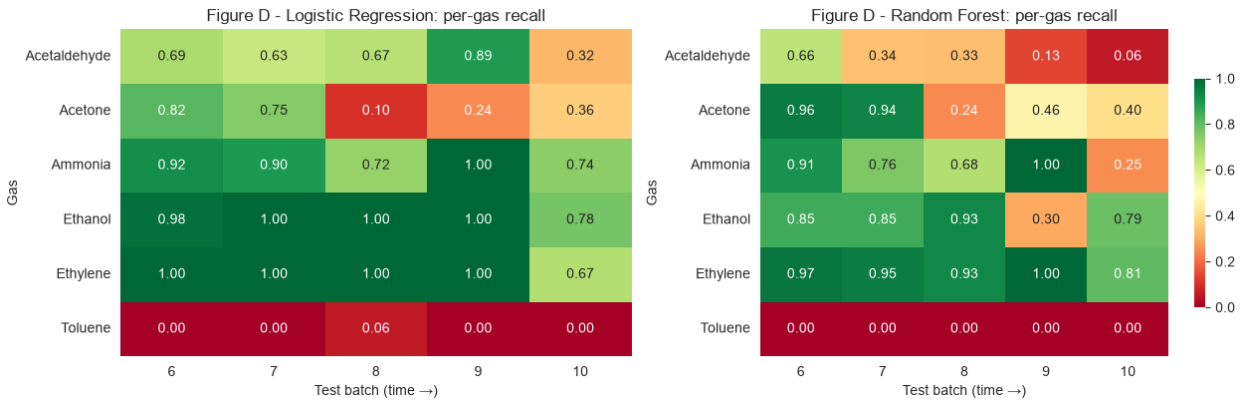



Figure 15. Per-gas recall by later batch. Blank cells indicate absent classes rather than zero recall.

A particularly serious failure is Toluene: recall falls close to zero in later batches for both models. Only about 79 Toluene samples occur in early batches 1-5, and batches 3-5 contain none. This limited historical coverage is a plausible contributing factor, not proof that sensor ageing or class scarcity alone caused the failure. Concentration distributions also vary across batches, and concentration was not included as an input.

## 6. Critical Discussion, Limitations and Future Improvements

**Model Comparison Summary**

Bringing the evidence together, the two chosen algorithms contrast clearly. On matched (random split) data Random Forest edges Logistic Regression on every metric; under temporal drift neither is consistently better (Logistic Regression wins two of three splits). Random Forest is almost unaffected by PCA, whereas PCA costs the linear model markedly; Logistic Regression is cheaper to train and more directly interpretable. Both far exceed a majority-class baseline of about 22% accuracy, confirming they capture real structure rather than guessing the most common gas.

| **Aspect** | **Logistic Regression** | **Random Forest** |
|---|---|---|
| Model family | Linear, parametric | Tree ensemble (bagging), non-parametric |
| Random-split test accuracy | 0.991 | 0.993 |
| Random-split macro-F1 | 0.991 | 0.993 |
| Drift accuracy (train 1-5 / test 6-10) | 0.625 | 0.557 |
| Sensitivity to PCA (CV macro-F1) | large drop (-0.080) | negligible (-0.004) |
| Interpretability | High (coefficients) | Moderate (feature importances) |
| Relative training cost | Low | Higher |
| Preferable when | Interpretability or robustness to shift matters | Maximum in-distribution accuracy matters |

**What worked**: the original standardized features with Random Forest gave the best recorded random-split results, and Logistic Regression was close behind while remaining easy to interpret. What did not help: engineered and selected features under-performed the raw set, and PCA reduced dimensionality substantially but hurt the linear classifier more than the ensemble.

**How performance might improve**: broaden each gas's coverage across acquisition periods; collect measurements at comparable concentrations; update models periodically as the sensor array changes; and test drift-detection, domain-adaptation or concentration-aware strategies [2,3]. Larger, nested hyper-parameter searches are possible but may add little relative to better temporal coverage.

- The data come from a single sensor array and laboratory.
- Some gas classes are absent from particular batches, and the temporal test configurations overlap.
- The future-batch splits were examined during development, so they are sensitivity analyses, not prospective tests.
- Engineered sensor summaries have no assured physical meaning.
- Historical class mix, concentration and sensor ageing are not separately controlled, so a large random-split-versus-drift gap demonstrates a generalization problem rather than a causal estimate of ageing alone.

**Conclusion**: Both algorithms classify randomly held-out measurements very well, but performance weakens markedly when models are trained on earlier batches and tested on later ones. The most meaningful finding is this evaluation gap, not a claim that one model is always more robust and it argues for temporal, class-wise evaluation before any deployment decision.

## 7. Individual Contribution

| Member | Primary Responsibilities |
|---|---|
| Saara M.K.F – MS26923024 | • Random Forest track<br>• PCA and dimensionality reduction<br>• Six-algorithm screening; sensor-drift experiments (chronological protocol, per-batch and per-gas analysis)<br>• Corresponding report sections. |
| Dhanushikan V – MS26924250 | • Logistic Regression track<br>• Data loading and audit; exploratory data analysis<br>• Standardization<br>• Feature engineering and selection (ANOVA, mutual information)<br>• Corresponding report sections. |

## 8. References

[1] Vergara, A. (2012). Gas Sensor Array Drift Dataset at Different Concentrations [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5MK6M (CC BY 4.0).

[2] Vergara, A., Vembu, S., Ayhan, T., Ryan, M. A., Homer, M. L., & Huerta, R. (2012). Chemical gas sensor drift compensation using classifier ensembles. Sensors and Actuators B: Chemical, 166–167, 320–329.

[3] Rodriguez-Lujan, I., Fonollosa, J., Vergara, A., Homer, M., & Huerta, R. (2014). On the calibration of sensor arrays for pattern recognition using the minimal number of experiments. Chemometrics and Intelligent Laboratory Systems, 130, 123–134.

[4] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[5] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

[6] Jolliffe, I. T. (2002). Principal Component Analysis (2nd ed.). Springer.

## 9. Appendix A - Complete Source Code

```
# Code Cell 1
import io, urllib.request
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import time


from sklearn.model_selection import (train_test_split, StratifiedKFold,
                                     cross_val_score, GridSearchCV, learning_curve)
from sklearn.preprocessing import StandardScaler, FunctionTransformer, label_binarize
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.feature_selection import f_classif, mutual_info_classif, SelectKBest
from sklearn.metrics import (accuracy_score, f1_score, balanced_accuracy_score,
                             precision_score, recall_score, roc_auc_score,
                             confusion_matrix, classification_report)


# Warnings are intentionally NOT globally silenced, so genuine issues
# (e.g. non-convergence) remain visible.
```

```
RANDOM_STATE = 42
sns.set_style("whitegrid")
%matplotlib inline
np.random.seed(RANDOM_STATE)

# Final models (reported). A lighter forest is used ONLY inside CV/tuning loops for speed;
# this is stated wherever it is used.
def make_lr():       return LogisticRegression(max_iter=2000, class_weight="balanced",
random_state=RANDOM_STATE)
def make_rf():       return RandomForestClassifier(n_estimators=200, min_samples_leaf=2,
                                    class_weight="balanced_subsample", n_jobs=-1, random_state=RANDOM_STATE)
def make_rf_light(): return RandomForestClassifier(n_estimators=80, min_samples_leaf=2,
                                    class_weight="balanced_subsample", n_jobs=1, random_state=RANDOM_STATE)

CV5 = StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE)
CV3 = StratifiedKFold(3, shuffle=True, random_state=RANDOM_STATE)

def multiclass_scores(y_true, y_pred):
    return {"Accuracy": accuracy_score(y_true, y_pred),
            "Macro-Prec": precision_score(y_true, y_pred, average="macro"),
            "Macro-Recall": recall_score(y_true, y_pred, average="macro"),
            "Macro-F1": f1_score(y_true, y_pred, average="macro"),
            "Balanced-Acc": balanced_accuracy_score(y_true, y_pred)}
```

---

**# Code Cell 2**

```
GAS = {1:"Ethanol", 2:"Ethylene", 3:"Ammonia", 4:"Acetaldehyde", 5:"Acetone", 6:"Toluene"}
BASE = "https://raw.githubusercontent.com/HakanKARASU/Gas-Sensor-Array-Drift-Dataset/main/"
FEATURES = [f"feature_{i}" for i in range(1, 129)]

def parse_dat(lines):
```

```
    labels, concs, feats = [], [], []
    for line in lines:
        line = line.strip()
        if not line:
            continue
        parts = line.split()
        cls, conc = parts[0].split(";")           # 'class;concentration'
        row = np.zeros(128)
        for tok in parts[1:]:
            idx, val = tok.split(":")
            row[int(idx)-1] = float(val)
        labels.append(int(cls)); concs.append(float(conc)); feats.append(row)
    return np.array(feats), np.array(labels), np.array(concs)

def load_batch(b):
    local = Path("data")/f"batch{b}.dat"
    if local.exists():
        with open(local) as f: lines = f.readlines()
    else:
        lines = urllib.request.urlopen(BASE + f"batch{b}.dat").read().decode().splitlines()
    X, y, conc = parse_dat(lines)
    dfb = pd.DataFrame(X, columns=FEATURES)
    dfb["gas"] = [GAS[c] for c in y]
    dfb["batch"] = b                          # true chronological batch from filename
    dfb["concentration"] = conc                   # METADATA only - never used as a predictor
    return dfb

df = pd.concat([load_batch(b) for b in range(1, 11)], ignore_index=True)
print("Loaded shape:", df.shape)
df.head(3)
```

**# Code Cell 3**

```
# Validate against the documented UCI ten-batch counts (total 13,910)

expected = {1:445, 2:1244, 3:1586, 4:161, 5:197, 6:2300, 7:3613, 8:294, 9:470, 10:3600}

counts = df["batch"].value_counts().sort_index().to_dict()

print("Per-batch counts match UCI documentation:", counts == expected)

print("Total rows:", len(df), "| Missing values:", int(df[FEATURES].isna().sum().sum()))
```

**# Code Cell 4**

```
print("Gas class counts:\n", df["gas"].value_counts(), "\n")

print("Feature scale (min .. max): {:.0f} .. {:.0f}".format(df[FEATURES].values.min(),
df[FEATURES].values.max()))

print("Most skewed features:\n", df[FEATURES].skew().abs().sort_values(ascending=False).head(3).round(1), "\n")

print("Class-by-batch (chronological):")

pd.crosstab(df["batch"], df["gas"])
```

**# Code Cell 5**

```
fig, ax = plt.subplots(1, 2, figsize=(11, 3.4))

vc = df["gas"].value_counts()

sns.barplot(x=vc.values, y=vc.index, hue=vc.index, palette="crest", legend=False, ax=ax[0])

ax[0].set_title("Gas class distribution"); ax[0].set_xlabel("Measurements")

bs = df["batch"].value_counts().sort_index()

sns.barplot(x=bs.index.astype(str), y=bs.values, hue=bs.index.astype(str), palette="flare", legend=False, ax=ax[1])

ax[1].set_title("Samples per batch (chronological \u2192)"); ax[1].set_xlabel("Batch");
ax[1].set_ylabel("Measurements")

plt.tight_layout(); plt.show()
```

**# Code Cell 6**

```
comp = (pd.crosstab(df["batch"], df["gas"], normalize="index") * 100)

comp = comp[sorted(comp.columns)]

comp.plot(kind="bar", stacked=True, figsize=(8, 4), colormap="tab10")

plt.ylabel("Class proportion within batch (%)"); plt.xlabel("Batch (chronological \u2192)")
```

```
plt.title("Figure B - Gas-class composition across acquisition batches")
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left", fontsize=8); plt.xticks(rotation=0)
plt.tight_layout(); plt.show()
```

**# Code Cell 7**

```
print("Median concentration (ppmv) by batch:")
print(df.groupby("batch")["concentration"].median().round(1).to_string())
plt.figure(figsize=(7.5, 3.4))
sns.boxplot(data=df, x="batch", y="concentration", hue="batch", palette="flare", legend=False, showfliers=False)
plt.title("Concentration distribution per batch (metadata only)")
plt.xlabel("Batch (chronological \u2192)"); plt.ylabel("Concentration (ppmv)")
plt.tight_layout(); plt.show()
```

**# Code Cell 8**

```
X = df[FEATURES].copy()
y = df["gas"].values
batch = df["batch"].values
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=RANDOM_STATE)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

**# Code Cell 9**

```
subset = [f"feature_{i}" for i in range(1, 25)]   # first 3 sensors x 8 features
plt.figure(figsize=(6.5, 5.2))
sns.heatmap(X_train[subset].corr(), cmap="coolwarm", center=0, square=True,
            cbar_kws={"shrink": .7}, xticklabels=False)
plt.title("Figure A - Pearson correlation, first 3 sensors (training set)")
plt.tight_layout(); plt.show()
```

**# Code Cell 10**

```
lr_noscale = Pipeline([("model", make_lr())])
lr_scaled  = Pipeline([("scale", StandardScaler()), ("model", make_lr())])
s0 = cross_val_score(lr_noscale, X_train, y_train, cv=CV3, scoring="accuracy", n_jobs=-1)
s1 = cross_val_score(lr_scaled,  X_train, y_train, cv=CV3, scoring="accuracy", n_jobs=-1)
print(f"LR CV accuracy WITHOUT scaling : {s0.mean():.4f} +/- {s0.std():.4f}")
print(f"LR CV accuracy WITH scaling    : {s1.mean():.4f} +/- {s1.std():.4f}")
print(f"Improvement from standardisation: {s1.mean()-s0.mean():+.4f}")
```

**# Code Cell 11**

```
def engineer_np(X):
    X = np.asarray(X, dtype=float)
    slog = np.sign(X) * np.log1p(np.abs(X))                    # 128
    sensor_means = np.stack([X[:, s*8:(s+1)*8].mean(1) for s in range(16)], axis=1)   # 16
    sensor_rng   = np.stack([X[:, s*8:(s+1)*8].max(1) - X[:, s*8:(s+1)*8].min(1) for s in range(16)], axis=1)  # 16
    denom = sensor_means.mean(1, keepdims=True)
    denom = np.where(np.abs(denom) < 1e-6, np.nan, denom)        # guard near-zero
    ratios = sensor_means / denom                              # 16
    ratios = np.nan_to_num(ratios, nan=0.0, posinf=0.0, neginf=0.0)
    return np.hstack([slog, sensor_means, sensor_rng, ratios])   # 176

print("Engineered feature count:", engineer_np(X_train.values).shape[1],
      "(128 signed-log + 16 means + 16 ranges + 16 ratios)")
# NOTE: these aggregates/ratios are *mathematical* summaries, not necessarily physical quantities
# (the 8 features per sensor describe different response characteristics). Their value is judged
# empirically by CV; in Section 11 the original 128 features actually outperform this set.
```

**# Code Cell 12**

```
eng_cols = ([f"slog_{i+1}" for i in range(128)] +
            [f"sensor{s+1}_mean" for s in range(16)] +
```

```
        [f"sensor{s+1}_range" for s in range(16)] +
        [f"sensor{s+1}_ratio" for s in range(16)])
Xtr_eng = engineer_np(X_train.values)
Xtr_eng_s = StandardScaler().fit_transform(Xtr_eng)

f_sc, _ = f_classif(Xtr_eng_s, y_train)
sub = np.random.RandomState(RANDOM_STATE).permutation(len(y_train))[:4000]   # subsample for MI speed
mi_sc = mutual_info_classif(Xtr_eng_s[sub], y_train[sub], random_state=RANDOM_STATE)
rank = pd.DataFrame({"feature": eng_cols, "ANOVA_F": f_sc, "MutualInfo": mi_sc})
rank["avg_rank"] = (rank["ANOVA_F"].rank(ascending=False) + rank["MutualInfo"].rank(ascending=False)) / 2
display(rank.sort_values("avg_rank").head(10).round(4))
```

**# Code Cell 13**

```
scaler_v = StandardScaler().fit(X_train)
pca_full = PCA().fit(scaler_v.transform(X_train))
cum = np.cumsum(pca_full.explained_variance_ratio_)
n95 = int(np.argmax(cum >= 0.95) + 1)

fig, ax = plt.subplots(1, 2, figsize=(11, 3.6))
ax[0].plot(range(1, len(cum)+1), cum, lw=2)
ax[0].axhline(0.95, color="r", ls="--", lw=1, label="95% variance")
ax[0].axvline(n95, color="g", ls="--", lw=1, label=f"{n95} components")
ax[0].set_xlabel("Components"); ax[0].set_ylabel("Cumulative explained variance")
ax[0].set_title("PCA explained variance"); ax[0].legend(fontsize=8)
p2 = PCA(n_components=2, random_state=RANDOM_STATE).fit_transform(scaler_v.transform(X_train))
for g in np.unique(y_train):
    m = y_train == g
    ax[1].scatter(p2[m,0], p2[m,1], s=6, alpha=.5, label=g)
ax[1].set_xlabel("PC1"); ax[1].set_ylabel("PC2"); ax[1].set_title("First two PCs (train)")
ax[1].legend(fontsize=7, markerscale=2)
```

```
plt.tight_layout(); plt.show()

print(f"{n95} components capture 95% of the training-set variance (NB: variance retained is NOT classification accuracy).")
```

**# Code Cell 14**

```
def pca_pipe(model):            # reused by later cells - keep defined
    return Pipeline([("scale", StandardScaler()),
                     ("pca", PCA(n_components=0.95, random_state=RANDOM_STATE)), ("model", model)])
def orig_pipe(model):
    return Pipeline([("scale", StandardScaler()), ("model", model)])


screen = {"Logistic Regression": make_lr(),
          "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=5),
          "Decision Tree": DecisionTreeClassifier(class_weight="balanced", random_state=RANDOM_STATE),
          "Random Forest": make_rf_light(),
          "Gaussian Naive Bayes": GaussianNB(),
          "SVM (RBF)": SVC(kernel="rbf", class_weight="balanced", random_state=RANDOM_STATE)}
rows = []
for name, m in screen.items():
    for rep_name, pipe in [("Original (128)", orig_pipe(m)), ("PCA (95%)", pca_pipe(m))]:
        sc = cross_val_score(pipe, X_train, y_train, cv=CV3, scoring="f1_macro", n_jobs=-1)
        rows.append({"Algorithm": name, "Representation": rep_name,
                     "CV Macro-F1": round(sc.mean(), 4), "CV std": round(sc.std(), 4)})
screen_df = pd.DataFrame(rows)
comparison = screen_df.pivot(index="Algorithm", columns="Representation", values="CV Macro-F1")
comparison["PCA - Original"] = (comparison["PCA (95%)"] - comparison["Original (128)"]).round(4)
comparison = comparison.sort_values("Original (128)", ascending=False)
display(comparison)
```

**# Code Cell 15**

```
comparison[["Original (128)", "PCA (95%)"]].plot(kind="bar", figsize=(10, 4.5), rot=20)
plt.ylim(0, 1.03); plt.ylabel("3-fold CV Macro-F1 (train)")
plt.title("Six algorithms: original 128 features vs PCA (95%)")
plt.legend(title="Representation", fontsize=8); plt.tight_layout(); plt.show()
```

**# Code Cell 16**

```
eng_rows = {}
for name, factory in [("Logistic Regression", make_lr), ("Random Forest", make_rf_light)]:
    pipe = Pipeline([("eng", FunctionTransformer(engineer_np)), ("scale", StandardScaler()),
                     ("select", SelectKBest(f_classif, k=40)), ("model", factory())])
    sc = cross_val_score(pipe, X_train, y_train, cv=CV3, scoring="f1_macro", n_jobs=-1)
    eng_rows[name] = round(sc.mean(), 4)
rep_compare = comparison.loc[["Logistic Regression", "Random Forest"], ["Original (128)", "PCA (95%)"]].copy()
rep_compare["Engineered+Select"] = pd.Series(eng_rows)
print("Representation comparison (training CV macro-F1):")
display(rep_compare.round(4))
```

**# Code Cell 17**

```
rep_compare.plot(kind="bar", figsize=(7.5, 4), rot=0); plt.ylim(0, 1.03)
plt.ylabel("CV Macro-F1 (train)"); plt.title("Feature representation vs performance (training CV)")
plt.legend(fontsize=8); plt.tight_layout(); plt.show()
```

**# Code Cell 18**

```
component_counts = [2, 5, 10, 20, 30]
sweep = {"Logistic Regression": [], "Random Forest": []}
for n in component_counts:
    for name, factory in [("Logistic Regression", make_lr), ("Random Forest", make_rf_light)]:
        pipe = Pipeline([("scale", StandardScaler()),
```

```
                ("pca", PCA(n_components=n, random_state=RANDOM_STATE)), ("model", factory())])
        sweep[name].append(cross_val_score(pipe, X_train, y_train, cv=CV3, scoring="accuracy", n_jobs=-1).mean())
plt.figure(figsize=(5.6, 3.6))
for name in sweep: plt.plot(component_counts, sweep[name], marker="o", label=name)
plt.xlabel("Number of PCA components"); plt.ylabel("CV accuracy (train)")
plt.title("Accuracy vs PCA dimensionality"); plt.legend(fontsize=8)
plt.tight_layout(); plt.show()
```

**# Code Cell 19**

```
grids = {
 "Logistic Regression": (pca_pipe(make_lr()), {"model__C": [0.1, 1, 10]}),
 "Random Forest":       (pca_pipe(make_rf_light()),
                         {"model__n_estimators": [100, 200], "model__max_depth": [None, 20],
                          "model__min_samples_leaf": [2]}),
}
best_estimators = {}   # keep the fitted best estimator (refit on full train), not just params
rows = []
for name, (pipe, grid) in grids.items():
    gs = GridSearchCV(pipe, grid, scoring="f1_macro", cv=CV3, n_jobs=-1, refit=True).fit(X_train, y_train)
    best_estimators[name] = gs.best_estimator_
    rows.append({"Model": name, "Best CV Macro-F1": round(gs.best_score_, 4), "Best params":
str(gs.best_params_)})
pd.DataFrame(rows)
```

**# Code Cell 20**

```
classes = sorted(np.unique(y))
yb = label_binarize(y_test, classes=classes)
final = {
 ("All (128)",   "Logistic Regression"): Pipeline([("scale", StandardScaler()), ("model", make_lr())]),
 ("All (128)",   "Random Forest"):       Pipeline([("scale", StandardScaler()), ("model", make_rf())]),
 ("PCA (tuned)", "Logistic Regression"): best_estimators["Logistic Regression"],
```

```
 ("PCA (tuned)", "Random Forest"):       best_estimators["Random Forest"],
}
fitted_final, rows = {}, []
for (rep_name, name), est in final.items():
    if rep_name.startswith("All"):        # best_estimators are already refit by GridSearchCV
        est.fit(X_train, y_train)
    fitted_final[(rep_name, name)] = est
    pred, proba = est.predict(X_test), est.predict_proba(X_test)
    rows.append({"Representation": rep_name, "Model": name,
                 "Accuracy": round(accuracy_score(y_test, pred), 4),
                 "Macro-F1": round(f1_score(y_test, pred, average="macro"), 4),
                 "Balanced-Acc": round(balanced_accuracy_score(y_test, pred), 4),
                 "ROC-AUC(ovr)": round(roc_auc_score(yb, proba, average="macro", multi_class="ovr"), 4)})
print("Final test results (All-128 = best CV representation; PCA = required reduction):")
final_table = pd.DataFrame(rows); display(final_table)
```

**# Code Cell 21**

```
mets = ["Accuracy", "Macro-F1", "Balanced-Acc"]
xpos = np.arange(len(final_table)); w = 0.25
plt.figure(figsize=(8.5, 4))
for k, met in enumerate(mets):
    plt.bar(xpos + k*w, final_table[met].values, w, label=met)
plt.xticks(xpos + w, [f"{r}\n{m}" for r, m in zip(final_table["Representation"], final_table["Model"])], fontsize=8)
plt.ylim(0, 1.03); plt.ylabel("Score (test)")
plt.title("Figure C - Final test performance by configuration"); plt.legend(fontsize=8)
plt.tight_layout(); plt.show()
```

**# Code Cell 22**

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4.3))
for ax, name in zip(axes, ["Logistic Regression", "Random Forest"]):
```

```
    est = fitted_final[("All (128)", name)]
    cm = confusion_matrix(y_test, est.predict(X_test), labels=classes)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, ax=ax,
                xticklabels=classes, yticklabels=classes)
    ax.set_title(f"{name} — All (128), test"); ax.set_xlabel("Predicted"); ax.set_ylabel("Actual")
    ax.tick_params(axis="x", rotation=45)
plt.tight_layout(); plt.show()
```

**# Code Cell 23**

```
rf_est = fitted_final[("All (128)", "Random Forest")].named_steps["model"]
lr_est = fitted_final[("All (128)", "Logistic Regression")].named_steps["model"]
imp    = pd.Series(rf_est.feature_importances_, index=FEATURES).sort_values(ascending=False).head(12)
lr_imp = pd.Series(np.abs(lr_est.coef_).mean(0), index=FEATURES).sort_values(ascending=False).head(12)
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
sns.barplot(x=imp.values, y=imp.index, hue=imp.index, palette="viridis", legend=False, ax=ax[0])
ax[0].set_title("Random Forest - top importances"); ax[0].set_xlabel("Importance")
sns.barplot(x=lr_imp.values, y=lr_imp.index, hue=lr_imp.index, palette="magma", legend=False, ax=ax[1])
ax[1].set_title("Logistic Regression - top |coef| (mean over classes)"); ax[1].set_xlabel("Mean |coef|")
plt.tight_layout(); plt.show()
```

**# Code Cell 24**

```
plt.figure(figsize=(5.8, 3.8))
for name, factory, color in [("Logistic Regression", make_lr, "tab:blue"),
                             ("Random Forest", make_rf_light, "tab:orange")]:
    ts, tr, va = learning_curve(pca_pipe(factory()), X_train, y_train,
                                train_sizes=np.linspace(0.1, 1.0, 4), cv=CV3,
                                scoring="accuracy", n_jobs=-1)
    plt.plot(ts, tr.mean(1), "--", color=color, alpha=.6)
    plt.plot(ts, va.mean(1), "-o", color=color, label=f"{name} (CV)")
plt.xlabel("Training examples"); plt.ylabel("Accuracy")
```

```
plt.title("Learning curves (dashed = train, solid = CV)"); plt.legend(fontsize=8)
plt.tight_layout(); plt.show()
```

**# Code Cell 25**

```
def drift_eval(train_batches, test_batches):
    tr = np.isin(batch, train_batches); te = np.isin(batch, test_batches)
    out = {}
    for name, factory in [("Logistic Regression", make_lr), ("Random Forest", make_rf)]:
        pipe = pca_pipe(factory()).fit(X.values[tr], y[tr]); pred = pipe.predict(X.values[te])
        out[(name, "Acc")]      = round(accuracy_score(y[te], pred), 4)
        out[(name, "Macro-F1")] = round(f1_score(y[te], pred, labels=classes, average="macro", zero_division=0), 4)
        out[(name, "Bal-Acc")]  = round(balanced_accuracy_score(y[te], pred), 4)
    return out

configs = {"Train 1-5 / test 6-10": ([1,2,3,4,5], [6,7,8,9,10]),
           "Train 1-3 / test 4-10": ([1,2,3],     [4,5,6,7,8,9,10]),
           "Train 1-6 / test 7-10": ([1,2,3,4,5,6],[7,8,9,10])}
drift_df = pd.DataFrame({k: drift_eval(*v) for k, v in configs.items()}).T
print("Later-batch performance. Context: these drift models use the PCA pipeline, whose random-split\n"
      "      TEST accuracy was ~0.936 (LR) / ~0.991 (RF) - a different protocol, not a matched baseline):")
display(drift_df)
wins = {k: ("LR" if drift_df.loc[k, ("Logistic Regression", "Acc")] >= drift_df.loc[k, ("Random Forest", "Acc")] else "RF")
        for k in configs}
print("Higher accuracy by configuration:", wins)
```

**# Code Cell 26**

```
# Per-batch degradation + train-vs-future gap (trained on batches 1-5)
tr = np.isin(batch, [1,2,3,4,5])
fitted = {name: pca_pipe(f()).fit(X.values[tr], y[tr])
          for name, f in [("Logistic Regression", make_lr), ("Random Forest", make_rf)]}
```

```
print("Train-vs-future gap (consistent with overfitting, but also with distribution shift):")
for name, m in fitted.items():
    tr_acc = accuracy_score(y[tr], m.predict(X.values[tr]))
    later = [accuracy_score(y[batch==b], m.predict(X.values[batch==b])) for b in [6,7,8,9,10]]
    sizes = [np.sum(batch==b) for b in [6,7,8,9,10]]
    print(f"  {name:22s} train={tr_acc:.3f}  later unweighted-
mean={np.mean(later):.3f}  pooled={np.average(later,weights=sizes):.3f}")
te15 = np.isin(batch, [6,7,8,9,10])
print("\nPer-class detail on later batches 6-10 (trained on 1-5):")
for name, m in fitted.items():
    print(f"\n--- {name} ---")
    print(classification_report(y[te15], m.predict(X.values[te15]), labels=classes, zero_division=0, digits=3))


plt.figure(figsize=(5.8, 3.6))
for name, m in fitted.items():
    plt.plot([6,7,8,9,10], [accuracy_score(y[batch==b], m.predict(X.values[batch==b])) for b in [6,7,8,9,10]],
             marker="o", label=name)
plt.xlabel("Test batch (later in time \u2192)"); plt.ylabel("Accuracy"); plt.ylim(0, 1)
plt.title("Performance under sensor drift (trained on batches 1-5)"); plt.legend(fontsize=8)
plt.tight_layout(); plt.show()
```

**# Code Cell 27**

```
from sklearn.metrics import recall_score
later_batches = [6, 7, 8, 9, 10]
fig, axes = plt.subplots(1, 2, figsize=(13, 4.2))
for ax, (name, m) in zip(axes, fitted.items()):
    M = np.full((len(classes), len(later_batches)), np.nan)
    for j, b in enumerate(later_batches):
        mk = batch == b; yt = y[mk]
        rec = recall_score(yt, m.predict(X.values[mk]), labels=classes, average=None, zero_division=0)
        present = set(yt)
```

```
        for i, g in enumerate(classes):
            if g in present: M[i, j] = rec[i]
    sns.heatmap(M, annot=True, fmt=".2f", cmap="RdYlGn", vmin=0, vmax=1, ax=ax,
                xticklabels=later_batches, yticklabels=classes, mask=np.isnan(M),
                cbar=(ax is axes[-1]), cbar_kws={"shrink": .7})
    ax.set_title(f"Figure D - {name}: per-gas recall"); ax.set_xlabel("Test batch (time \u2192)"); ax.set_ylabel("Gas")
plt.tight_layout(); plt.show()
```

---